COMPUTER ASSIGNMENT - 1

ANALYZING 1D HEAT CONVECTION IN FLAT PLATE WITH EXPLICIT METHOD

Problem Definition:

The objective of this study is to solve the 1D heat flow through a flat plate using the convection equation with the explicit finite difference method. This approach aims to determine the temperature distribution along the length of the plate over various time intervals.

Governing Equation:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}$$

T- Temperature

t – Time

x – Spatial Coordinate

∝- Thermal Diffusivity

Initial Condition:

All temperatures are set to zero initially.

Boundary Conditions:

- At x= 0 Temperature is fixed T = 1.0
- At x = L Temperature is fixed T = 0.0

Numerical Formulation:

Forward Difference in Time and Central Difference in Spatial

Explicit Method

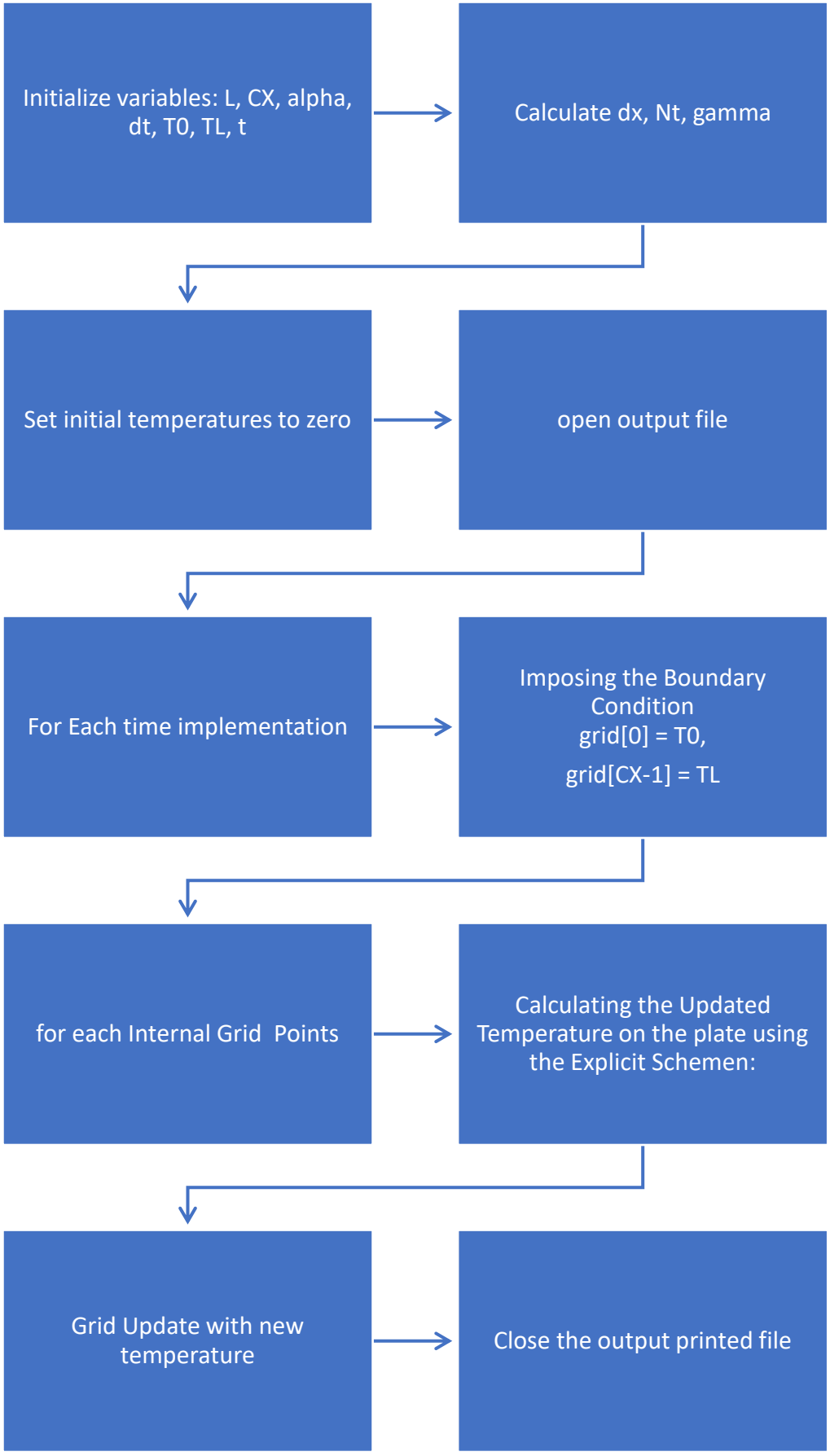
$$T_i^{n+1} = \gamma T_{i-1}^n + (1 - 2\gamma) T_i^n + \gamma T_{i+1}^n$$

$$\gamma = \frac{\Delta t}{\Delta x^2}$$

$\Delta t -$ *the time step*

$\Delta x^2 -$ the spatial step size

Flow Chart:


Initialize variables: L, CX, alpha, dt, T0, TL, t
Calculate dx, Nt, gamma
Set initial temperatures to zero
open output file
For Each time implementation
Imposing the Boundary Condition
grid[0] = T0,
grid[CX-1] = TL
for each Internal Grid Points
Calculating the Updated Temperature on the plate using the Explicit Schemen:
Grid Update with new temperature
Close the output printed file

Computer Code:

```
// 1D heat convection Equation
#include <stdio.h>
#include <math.h>
#define CX 51   // Number of grid points

int main() {
   // Declare the variables
   int L = 1; // m
   double grid[CX];
   double alpha = 1;       // m²/s
   double dt = 0.001;        // sec
   double T0 = 1.0;        // Boundary condition at x=0 length
   double TL = 0.0;        // Boundary condition at x=L length
   double TN[CX];          // Temperature at new grid points
   double t = 10.0;        // Total time for simulation
   int DX,DT; // for output file notation

   // Cell size calculation
   double dx;              // Size of the cell in x-axis
   dx = L / (CX - 1);
   printf("dx: %lf\t", dx);
   printf("dt: %lf\t", dt);

   // Time steps calculations
   int Nt = (int)(t / dt);  // Number of time steps, cast to int for loop control
   printf("Nt: %d\t", Nt);

   // Gamma Calculation
   double gamma = alpha * (dt / (dx * dx));
   printf("\ngamma: %f\t", gamma);
```

```
  // Initial condition: Set all temperatures to zero initially.
for (int i = 0; i < CX; i++) {
    grid[i] = 0.0;
  }
  printf("\nInitial Temperature at grid Points:\n");
  for (int i = 0; i < CX; i++) {
    printf("grid[%d] = %.2f\n", i, grid[i]);
  }

        // Open a file for writing
  FILE *outputFile;
  outputFile = fopen("temperature_output.txt", "w");

  if (outputFile == NULL) {
    printf("Error opening file!\n");
    return 1;
  }

  // Write header to the file
        fprintf(outputFile, "DX\tDT\n"); // Header row


        // Time loop initializing
        for (int j = 0; j <= Nt; j++) {
                double Time = j * dt; // Calculating the run time
                printf("\nTime: %.2f Seconds\n", Time);

                // Impose boundary conditions
                grid[0] = T0;       // Left side Boundary condition
                grid[CX - 1] = TL;  // Right side Boundary Condition
```

```
                for (int i = 1; i < CX - 1; i++) {
                        TN[i] = (gamma * grid[i - 1]) + ((1 - (2 * gamma)) * grid[i]) + (gamma * grid[i +
1]);
                        printf("\nTN[%d] = %.2f\n", i, TN[i]);
                }

                for (int i = 1; i < CX - 1; i++) {
                        grid[i] = TN[i]; // Update temperature values for next iteration
                }

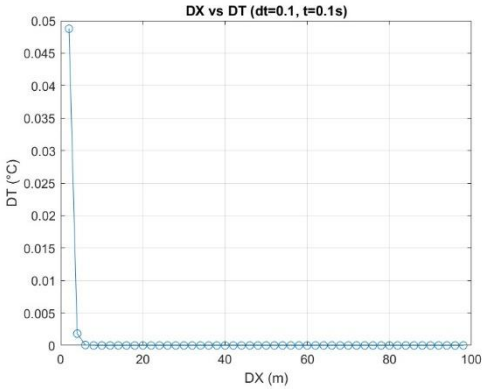
        // Write grid points and temperatures to the file after updating temperatures
        fprintf(outputFile, "Time: %.2f seconds\n", Time);
        for (int i = 0; i < CX; i++) {
      //    fprintf(outputFile, "Grid%d: Temperature %.2f\n", i, grid[i]);
            fprintf(outputFile, "%d\t%.2f\n", i, grid[i]);
        }
        fprintf(outputFile, "\n"); // New line for separation between time steps
             }

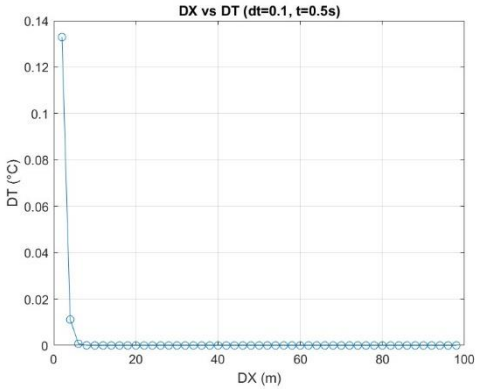
             fclose(outputFile); // Close the file after writing

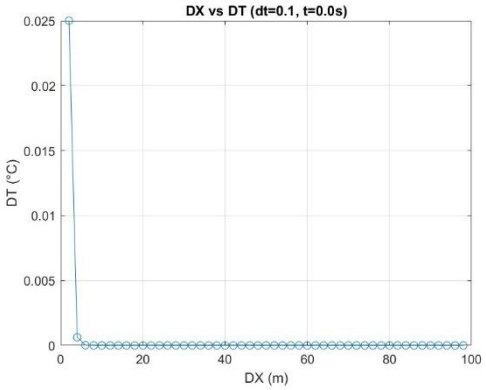
             return 0;
}
```

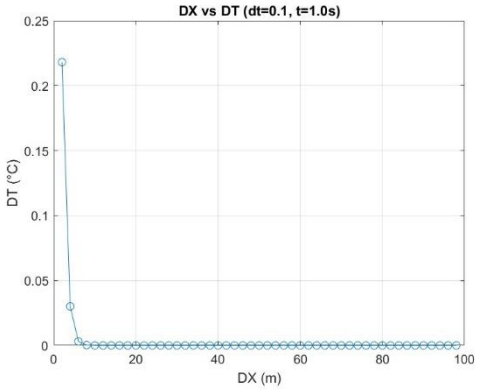
Plots:

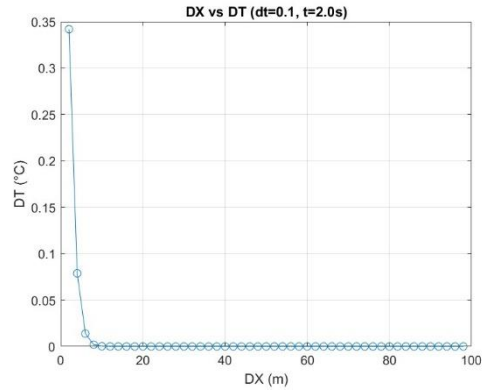
Plot for the dt = 0.1 s and t = 0, 0.1, 0.5, 1.0, 2.0, 10.0

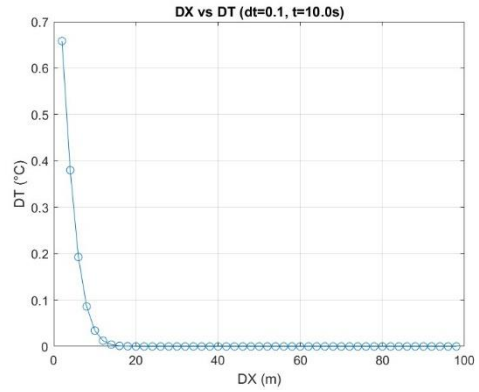




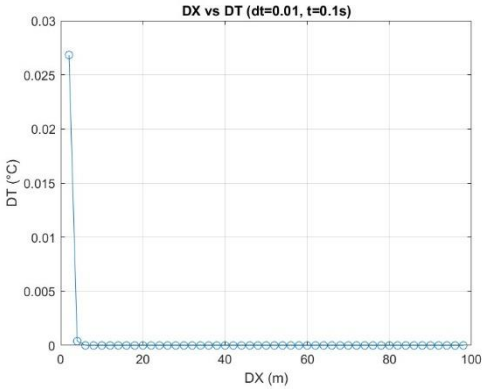


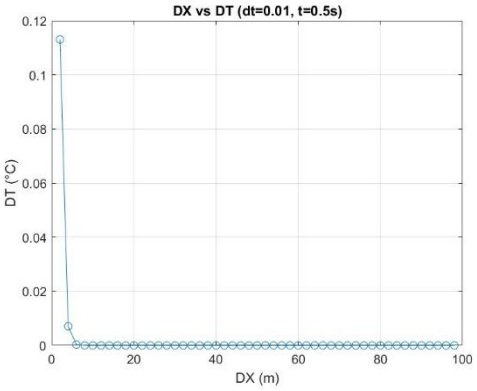


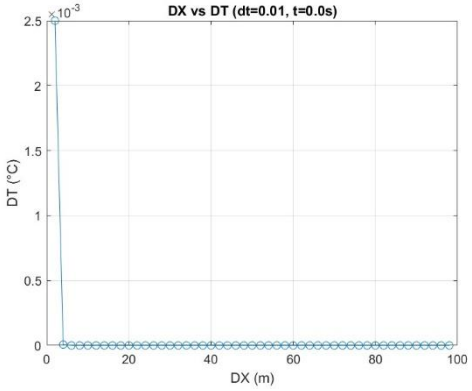


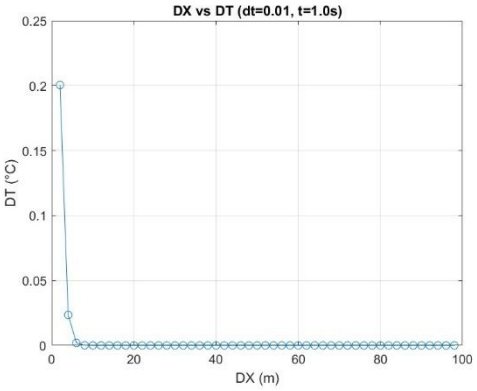


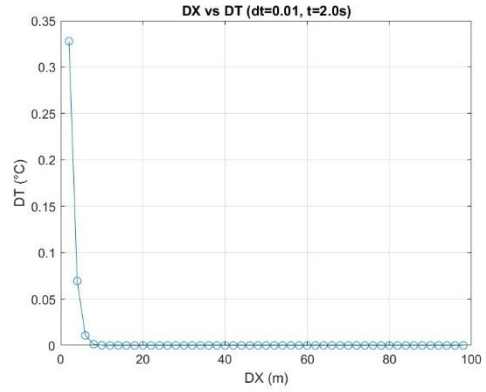
Plot for the dt = 0.01 s and t = 0, 0.1, 0.5, 1.0, 2.0, 10.0

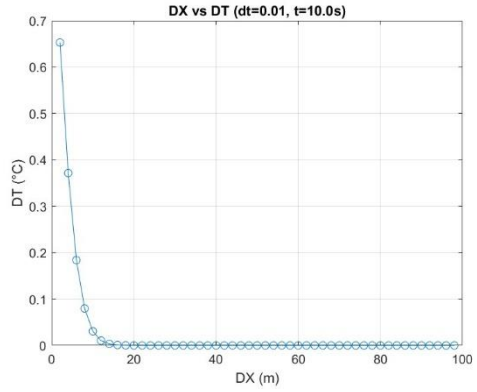




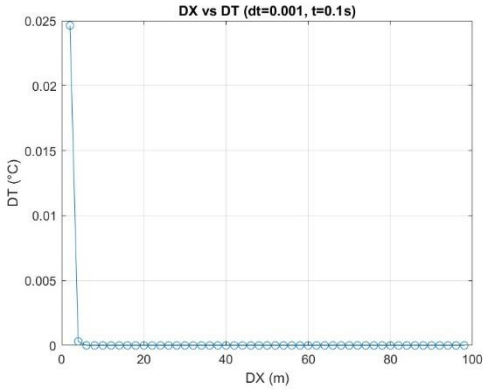


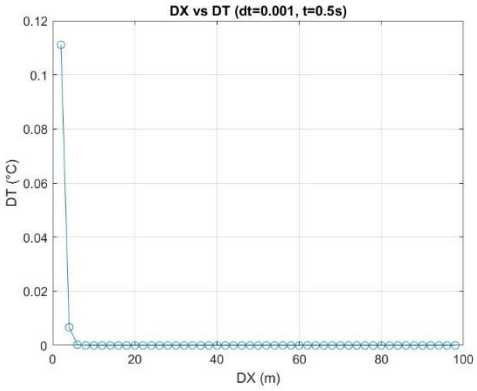


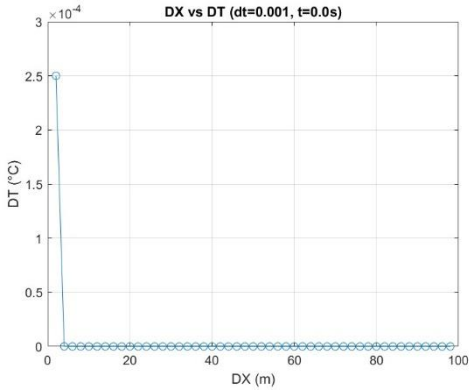


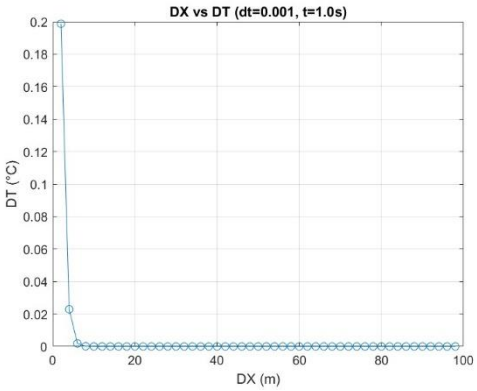


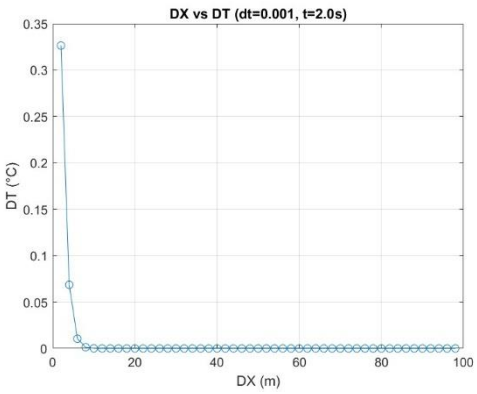
Plot for the dt = 0.001 s and t = 0, 0.1, 0.5, 1.0, 2.0, 10.0

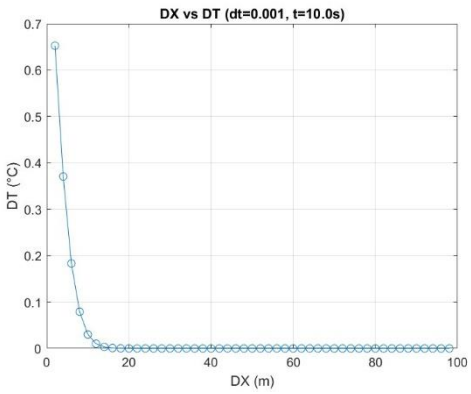












Result:

The implementation of heat convection through a one-dimensional flat plate using explicit finite difference methods has yielded accurate results. The simulation effectively demonstrates how thermal energy is transferred within a medium, providing insights into heat conduction principles. The plotted results validate that the numerical method used is capable of accurately capturing the dynamics of heat transfer in this scenario.